ResQCloud: AWS Account and Project Setup

Individual account setup, local development, AWS service integration, and project readiness references.

Prepared from the repository documentation. Review linked AWS service pricing and current account offers before creating resources.

Documentation index

Source: docs/README.md

ResQCloud documentation

Use this index to find setup instructions, implementation references, project status, and design material.

Setup and AWS

- AWS and project setup bundle: start here for an individual AWS account, local project setup, and optional AWS integrations.
- Compiled setup PDF: the combined account, project, AWS, and readiness guides.
- Local and individual account setup
- SQS first demo
- Full AWS service showcase
- Aurora PostgreSQL
- AWS architecture

Engineering references

- Application architecture
- API reference
- Authentication
- Database
- Deployment boundary
- Disaster recovery
- Events and failure handling
- Incident workflow
- Monitoring
- Notifications
- Operations and troubleshooting
- Authorization
- Security and privacy

- Aurora connectivity

Project and readiness

- Feature map and remaining work
- Implementation plan
- Requirement assumptions
- Implementation status and release gates
- Test results
- Regional disaster-readiness audit

Design

- UI refresh notes
- Design screenshots

For the repository overview, commands, and development setup, see the root README.

AWS setup guide index

Source: docs/aws/README.md

ResQCloud AWS and project setup

This folder contains the compiled PDF and its documentation entry point. The guide is for an individual developer setting up a local ResQCloud demo, with optional AWS integrations. The web application and API remain local throughout these instructions.

Start here

1. Follow the individual AWS account and local project setup. It covers account security, local prerequisites, environment setup, demo accounts, and both AWS paths.
2. Choose the level of AWS integration you need:

- SQS first demo for a single queue and scoped permissions.
- Full AWS service showcase for the multi-service configuration and cleanup steps.
- Aurora PostgreSQL details for database access, TLS, and IAM authentication.

1. Review the AWS architecture, deployment boundary, and implementation status before adapting the demo or treating it as production software.
2. Use the project README for repository commands and general development notes.

Contents

File	What it covers
Compiled PDF	Account setup, local project setup, AWS walkthroughs, and project readiness references in one document.
Individual account setup	Step-by-step setup for a personal AWS account, local development, SQS, and the full AWS demo.
SQS first demo	Queue creation, least-privilege access, verification, and cleanup.
Full AWS showcase	Console steps for Aurora, SQS, S3, SNS, and optional Cognito or CloudFront.
Aurora PostgreSQL	Database configuration, TLS, IAM database authentication, and connection checks.
AWS architecture	AWS resources and how they connect to the local app.
Deployment boundary	Current deployment model and constraints.
Implementation status	Current capabilities, known gaps, and production readiness notes.
Project README	Repository overview, commands, and development workflow.

Rebuild the PDF

From the repository root, run:

```
node scripts/renderAwsSetupPdf.mjs
```

The renderer reads the source files listed in `scripts/renderAwsSetupPdf.mjs` and writes the PDF into this folder. It uses the installed Playwright package and local Chromium; set `CHROMIUM_PATH` if Chromium is installed somewhere else.

Safety notes

Use fictional data only. The demo is not an emergency dispatch service, and its SOS action does not contact emergency services. AWS resources can incur charges; check current prices and account offers before creating them. A budget alert does not cap spending. Follow the source guide's cleanup checklist when finished.

Individual AWS account and project setup

Source: docs/aws/individual-setup.md

ResQCloud setup for an individual AWS account

This guide takes you from a new personal AWS account to a running local copy of ResQCloud. It separates the free-of-cloud local setup from optional AWS integrations so you can stop before creating billable resources.

ResQCloud's web app and API run on your computer in the documented AWS demo. AWS services provide selected data and messaging features; this process does not deploy the app to AWS. The application is a development demo, not an emergency dispatch service. Use fictional data only. Its SOS action does not contact emergency services.

Pick a setup path

Path	What you set up	AWS resources
Local first (recommended)	Web app, API, SQLite, fictional sample accounts	None
SQS demonstration	Local app plus AWS event queue	One SQS queue
Full AWS service demonstration	Local app plus Aurora PostgreSQL, SQS, S3, SNS; optional CloudFront and Cognito	Multiple AWS resources; may cost money

Finish the local path before deciding whether you need AWS resources. A budget alert is not a spending limit, and free-tier eligibility and prices depend on the account, region, and current AWS offer.

1. Create and secure an individual AWS account

If you already have an account, start at step 3.

1. Go to aws.amazon.com and choose **Create an AWS Account**.
2. Enter an email address that you control and choose a strong, unique root password. This email becomes the account's root sign-in identity.
3. Choose **Personal** for account type and enter your legal contact details. AWS may ask for phone and payment-card verification. Follow the on-screen verification steps.
4. Choose the support plan that suits you. A paid support plan is optional for this demo; review its current price before selecting it.
5. Sign in to the AWS Console as the **root user** using the account email. Root is for initial account configuration and account-owner tasks only.
6. In the account menu, record the 12-digit AWS account ID. You will use it to confirm CLI identity and form a narrow IAM policy.

7. Open **IAM → Users → root user → Assign MFA device** (the exact menu can vary). Enroll a passkey/security key or authenticator app, save recovery information somewhere secure, then sign out and verify MFA sign-in works.
8. Do **not** create root access keys. Do not use root for CLI commands or run the application as root.
9. Open **Billing and Cost Management → Billing preferences** and enable the billing alerts/reports you want. If available, create an AWS Budget with a small monthly amount and email alerts. A budget reports estimated usage; it does not block charges, and notices may be delayed.
10. Open **Billing → Free Tier** and review the offers and remaining usage available to this account. Do not assume an AWS service is free based on an old tutorial.

For an individual account, this guide uses a dedicated IAM user for the local AWS CLI profile. If AWS offers your account a different guided identity method, use an equivalent dedicated non-root identity with MFA and only the permissions needed for the chosen demo. Never use root credentials in the application.

2. Choose a region and prepare your computer

Use one AWS Region for every resource. Project examples use **Asia Pacific (Sydney),** `ap-southeast-2`. Regions have different availability and prices. Choose another Region only if you also change every Region-specific command, ARN, and `.env` value below.

Install these tools:

- Git
- Node.js 24.x and npm (verify with `node --version` and `npm --version`)
- AWS CLI v2.32.0 or later only if you plan to use AWS
- OpenSSL only if you opt into CloudFront signed evidence delivery

Clone the repository or open your existing checkout. From the project root, install dependencies:

```
npm ci
```

The local one-command demo creates a separate `.env.demo` and SQLite database in the repository root. It leaves any existing `.env` and `resqcloud.sqlite` untouched. For the optional AWS paths later in this guide, `node scripts/setupLocal.mjs` creates `.env` only if it does not already exist. Both environment files contain private random development secrets; keep them out of Git and browser variables.

3. Run and sign in to the local demo (no AWS account needed)

One command prepares the local SQLite database, creates or updates the fictional sample accounts/incidents, and starts both servers:

```
npm run demo
```

Open http://localhost:3000 and choose Citizen, Dispatcher, Responder, Supervisor, or Administrator to enter with a ready-made sample account. The API health endpoint is http://localhost:4000/health; `/ready` also checks the database. The same accounts are available for manual password sign-in: `citizen@resqcloud.test`, `dispatcher@resqcloud.test`, `responder@resqcloud.test`,

`supervisor@resqcloud.test`, and `admin@resqcloud.test`. Their shared password is `DEV_SEED_PASSWORD` in `.env`. The development interface uses one sample state, Tamil Nadu.

To stop both services, press **Ctrl+C** in the terminal running `npm run demo`. To rebuild the local database, use `npm run demo:reset`; it asks you to type `RESET` before deleting only `resqcloud-demo.sqlite`, reseeding, and restarting. This permanently deletes data in the demo database. The launcher will not overwrite `.env`, use PostgreSQL, or connect to AWS services.

Before continuing, you can run the project's checks:

```
npm test
npm run typecheck
npm run build
```

Browser tests are optional and need a browser installation; see the Playwright configuration and README.

4. Create a dedicated AWS CLI identity (only for AWS paths)

Do this while signed into the console as account owner for IAM setup. The app itself will use a separate identity.

1. In the AWS Console, set the Region selector to `ap-southeast-2`.
2. Open **IAM** → **Users** → **Create user** and enter `resqcloud-demo`.
3. Enable console access if the console offers it. Set a unique password; require password reset at first sign-in if you will use this identity interactively.
4. Add MFA for this user and verify it works.
5. Grant the AWS-managed `SignInLocalDevelopmentAccess` policy so AWS CLI browser sign-in can obtain temporary credentials. The service-specific permissions are added separately in the path you choose below. Do not attach broad administrator access for normal app use.
6. Install or update AWS CLI. Confirm the version:

```
aws --version
```

1. Sign in through the browser and confirm which account the CLI will use:

```
aws login --profile resqcloud-demo --region ap-southeast-2
aws sts get-caller-identity --profile resqcloud-demo
```

Check that `Account` is your 12-digit account ID and the identity is the dedicated demo identity. The SDK reads temporary credentials from this CLI profile; do not paste AWS credentials into `.env`. After changing IAM permissions, sign in again to refresh the temporary session.

If `aws login` is not available, update to the CLI version above. Do not work around it by creating root access keys.

5. Choose an AWS integration path

Option A: SQS only

This is the smallest AWS integration. It leaves SQLite in place and creates one Standard SQS queue. Follow SQS-first AWS demo setup from its “Create one SQS queue” step if the local project is already running, or follow it from the beginning for the complete queue walkthrough. It includes the exact queue settings, scoped IAM policy, `.env` variables, verification, and cleanup.

At a high level, the queue path is:

1. Create a Standard queue named `resqcloud-demo-events` in `ap-southeast-2`; use a 20-second receive wait and 60-second visibility timeout.
2. Add only `sqs:SendMessage`, `sqs:ReceiveMessage`, and `sqs:DeleteMessage` on that queue to the demo identity.
3. Set `AWS_REGION` and `SQS_QUEUE_URL` in `.env` using the exact queue URL from AWS.
4. Sign in again with `aws login`, then start `AWS_PROFILE=resqcloud-demo npm run dev`.
5. Create a fictional incident, verify in-app notifications and SQS CloudWatch metrics, then delete the queue and remove its permissions when finished.

Option B: Aurora, SQS, S3, and SNS

This path replaces local SQLite with Aurora PostgreSQL and creates several AWS resources. Review the current service prices and account-specific free offers before creating anything. The project's console-managed AWS showcase guide provides exact console steps, policy JSON, environment values, test flow, and deletion steps. The Aurora connection guide provides the database role SQL, TLS CA setup, IAM database authentication details, and a full-configuration alternative.

Use this order:

1. Complete the local demo and AWS CLI identity steps above.
2. In Aurora/RDS, create Aurora PostgreSQL only after reviewing the account's price/offer. The guide recommends **Create with express configuration** where eligible. Wait for **Available** and record the writer endpoint and cluster resource ID.
3. Create the `resq_app` database role with the SQL in Aurora PostgreSQL.
4. Create the SQS queue, private S3 bucket, and SNS topic using the AWS showcase. Keep S3 Block Public Access enabled. Confirm the SNS email subscription.
5. Add the showcase guide's runtime policy, substituting your own account ID, queue ARN, bucket name, topic ARN, and Aurora cluster resource ID. Check every substituted value before saving.
6. Download the RDS CA bundle as described in Aurora PostgreSQL. Set `.env` to use PostgreSQL, verified TLS, IAM database authentication, and the exact service names/URLs. Do not put a database password or AWS keys in the `.env` for express IAM authentication.
7. Optional: configure Cognito and/or CloudFront only by following the corresponding sections in the AWS showcase. Cognito's browser app client must have no client secret and must use authorization code with PKCE. Keep CloudFront private-key material local.
8. Run `npm run db:migrate` and `npm run db:seed` while `NODE_ENV=development`. The seed script refuses to run in production.

9. Run `aws login --profile resqcloud-demo --region ap-southeast-2`, then `AWS_PROFILE=resqcloud-demo npm run dev`.
10. Check `/health` and `/ready`, exercise integrations with fictional records, and read the showcase troubleshooting notes if a service is unavailable.
11. When finished, use the showcase guide's complete cleanup checklist. Check RDS snapshots, CloudFront, S3 contents, and Billing for remaining resources.

Do not use a real emergency report, citizen identity, or medical information in this demo. Aurora may be reachable from your computer in the express setup; AWS credentials alone do not provide a network route to a private database. Do not make a database public or open a security group to `0.0.0.0/0` to solve connectivity problems.

6. Project configuration reference

The root `.env.example` documents every setting. For local SQLite, `setupLocal.mjs` prepares the correct values. For AWS, edit the ignored root `.env` and restart the API after changes. `NEXT_PUBLIC_` *variables are embedded in browser code; rebuild the web app after changing them. Never put private keys, passwords, AWS credentials, or database tokens in* `NEXT_PUBLIC_` variables.

Variable	Purpose
`NODE_ENV`, `PORT`, `APP_ORIGIN`	Runtime mode, API port, and allowed web origin
`JWT_SECRET`, `DEV_SEED_PASSWORD`	Local application signing and seed-account passwords
`DB_DIALECT`, `DATABASE_URL`, `DB_SSL`, `DB_SSL_CA_PATH`, `DB_IAM_AUTH`, `SQLITE_PATH`	SQLite or PostgreSQL/Aurora configuration
`AWS_REGION`, `AWS_ENDPOINT`	AWS SDK Region; endpoint override is for S3-compatible local services, normally blank for AWS
`SQS_QUEUE_URL`, `S3_BUCKET`, `SNS_TOPIC_ARN`	Optional AWS integrations
`CLOUDFRONT_DOMAIN`, `CLOUDFRONT_PUBLIC_KEY_ID`, `CLOUDFRONT_PRIVATE_KEY_PATH`	Optional signed private-evidence delivery
`COGNITO_USER_POOL_ID`, `COGNITO_CLIENT_ID`, `NEXT_PUBLIC_COGNITO_CLIENT_ID`, `NEXT_PUBLIC_COGNITO_DOMAIN`	Optional hosted sign-in; browser client ID must match the API client ID
`NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_MAPBOX_TOKEN`	Browser API endpoint and optional map display

The app and API remain local in these instructions. The project has no AWS compute deployment or Terraform setup. See deployment boundary and implementation status before treating this project as production-ready.

7. Troubleshooting and teardown

Symptom	Check
`npm run db:seed` says the seed password is missing	Confirm `DEV_SEED_PASSWORD` exists in root `.env` and is at least 12 characters.

`aws` command is missing or `aws login` is unknown	Install/update AWS CLI v2.32.0 or later; check `aws --version`.
AWS `AccessDenied`	Confirm the selected profile/account, allowed service actions/resource ARN, Region, and refreshed CLI login. Check whether account-level policy restricts that service.
Database connection timeout	Check Aurora state, endpoint/Region/port, TLS CA path, and whether your network can reach the endpoint. A private VPC endpoint cannot be reached from a laptop without an approved network connection.
TLS certificate verification error	Use the current CA bundle and exact writer endpoint. Keep `DB_SSL=true`; do not turn off certificate validation.
Port 3000 or 4000 is busy	Stop the existing app process or configure another port/origin consistently.
Local app works but AWS integration does not	Check API logs and `/ready`; installed AWS SDK packages do not mean AWS resources or permissions exist.

For SQS, use the SQS guide's focused cleanup. For the multi-service path, follow the full AWS showcase cleanup checklist. Also run `aws logout --profile resqcloud-demo` to end the local CLI session. Removing `.env` or local database files is optional and permanently removes local secrets/data; decide what you want to keep before deleting them.

Related project guides

- Project README and commands
- SQS-first demo
- Full console-managed AWS showcase
- Aurora PostgreSQL setup
- AWS architecture
- Deployment boundary
- Implementation status

Project README

Source: README.md

ResQCloud

Incident coordination from first report to resolution

ResQCloud is a demonstration platform for coordinating community incident reports. A person submits a report, an authorized operations team reviews and assigns it, a responder records progress, and a supervisor closes the incident. The reporter can follow the report and leave feedback.

This repository is also an AWS integration demonstration. It connects a locally running web app and API to AWS-managed services for the database, event delivery, private evidence, alerts, and optional sign-in. The application is **not hosted on AWS** by this project. Cloud services must be configured in an AWS account before those integrations can be used; local SQLite mode is available for development and for demonstrations without cloud access.

For a teammate: start with the overview and setup below. For console-by-console AWS setup, see the AWS showcase guide. For what is implemented versus still unverified, see implementation status.

For the full user-facing feature inventory and prioritized gaps, see the feature map.

What the product does

The product follows a report through an auditable response workflow:

1. A citizen reports an incident, optionally anonymously, and receives a private tracking receipt.
2. A dispatcher reviews the report, records a reason, confirms its details and priority, and assigns a responder.
3. A responder accepts the assignment, updates response progress and records resolution.
4. A supervisor reviews the outcome and closes the incident.
5. The citizen follows status changes and can submit feedback after closure.

The interface has role-based workspaces for citizens, responders, dispatchers, supervisors, and administrators. The API enforces role, ownership, and department access; hiding a link in the interface is not the security control. Reports and transitions are validated, written through database transactions, and recorded in incident history and an event outbox.

This is a faculty/demo platform using fictional data. It is not an emergency dispatch service: the urgent in-app report creates a critical incident inside ResQCloud and does **not** call local emergency services.

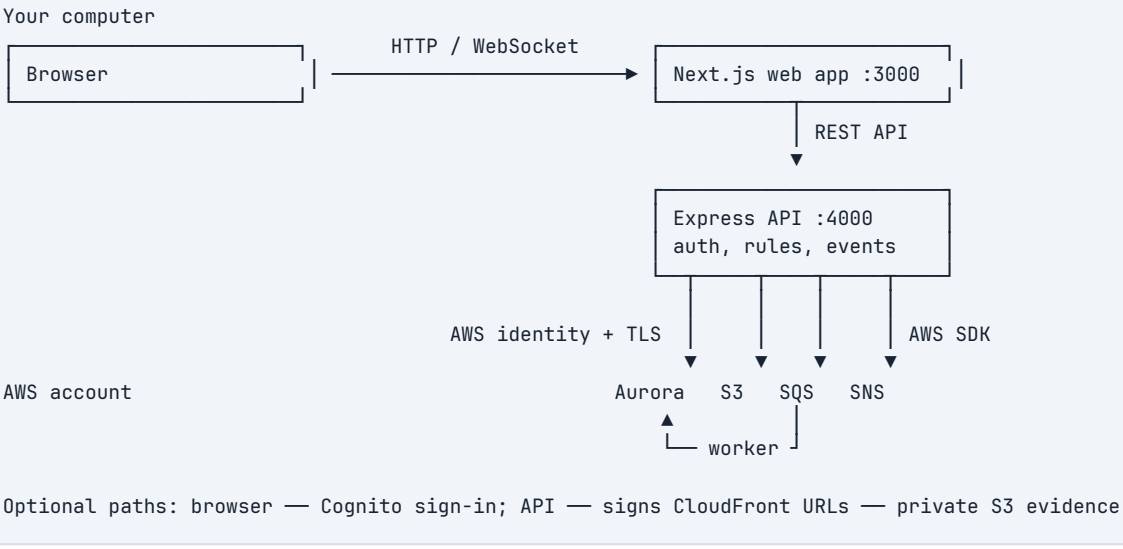
AWS connection, in plain language

Think of the web app and API as the part your teammate sees and runs. AWS supplies selected managed building blocks behind them:

AWS service	Plain-language role	How ResQCloud uses it
Aurora PostgreSQL	Managed relational database	Stores users, reports, assignments, status history, notifications, and queued events.
Amazon SQS	Durable work queue	Carries incident events from the API to a background consumer so event processing can retry independently.
Amazon S3	Private object storage	Holds incident evidence such as images. The bucket should remain private.
Amazon SNS	Notification publisher	Publishes critical-incident notifications to configured subscribers, such as a confirmed email subscription.
Amazon Cognito _(optional)_	Hosted identity service	Provides citizen sign-in and registration. The API validates Cognito ID tokens before creating an app session.
Amazon CloudFront _(optional)_	Content delivery network	Delivers private evidence with short-lived signed URLs while S3 remains private.
Mapbox _(optional, not AWS)_	Map display service	Displays incident locations when a public browser token is configured.

The AWS services are separate resources in an AWS account. ResQCloud does not create them automatically and does not use Terraform. They are configured in the AWS Console using the AWS showcase guide.

What runs where



The local AWS CLI profile supplies temporary AWS credentials to the API's AWS SDK clients. Database access is a separate connection: Sequelize connects to the Aurora PostgreSQL writer endpoint over TLS, and the API can generate a short-lived IAM database authentication token for the configured database user. The token is not a database password stored in `.env`.

Technical event and data flow

1. The browser sends requests to the Next.js app and its API client calls `http://localhost:4000/api/v1`. Protected requests carry a short-lived access token; a same-site, HttpOnly refresh cookie is rotated by the API.

2. Express validates input with shared Zod schemas, authenticates the actor, checks authorization and performs the incident operation.
3. Sequelize stores the change in Aurora PostgreSQL. A domain event is inserted in the same database transaction through the transactional outbox.
4. A single API worker sends outbox events to SQS when configured. Without SQS, local event delivery handles the event directly. The queue consumer validates messages and creates idempotent in-app notifications.
5. Critical events can be published to SNS by the event delivery handler, whether delivery is local or follows SQS consumption. The API sends authorized update notices to a user's WebSocket room; clients then reload data through the authorized REST API.
6. Evidence uploads use a short-lived, size- and content-type-restricted S3 upload policy. Optional CloudFront delivery requires an API authorization check and a five-minute signed URL.

The current worker is designed for one application instance. Queue delivery and SNS publication are at-least-once; notification records guard against duplicate processing where implemented. See architecture, events, and security for implementation details.

Quick start

Requirements

- Node.js 24 or newer and npm.
- Python 3, `make`, and a C++ compiler may be needed if the SQLite native package has no prebuilt binary for your platform.
- No AWS account is required for local development.

Run locally with SQLite

From the repository root:

```
npm ci
npm run demo
```

`npm run demo` creates a separate private `.env.demo` if needed, checks that it selects SQLite, seeds the schema and fictional sample data, then starts the web app and API. It uses a separate `resqcloud-demo.sqlite` database and leaves your existing `.env` and local database untouched. Local demo mode disables AWS integrations for this run and binds both servers to loopback. Press **Ctrl+C** to stop both.

Open http://localhost:3000. The Next.js web app uses port **3000**; the Express API uses port **4000**. The combined `npm run dev` command starts both. If startup reports that port 3000 is already in use, stop the other app process first; this project's web script is configured for 3000.

Seeded development accounts are:

Role	Email
Citizen	`citizen@resqcloud.test`
Dispatcher	`dispatcher@resqcloud.test`

Responder	`responder@resqcloud.test`
Supervisor	`supervisor@resqcloud.test`
Administrator	`admin@resqcloud.test`

When started with `npm run demo`, choose any role directly on the sign-in screen. This local shortcut is enabled only for the loopback SQLite development launcher. For manual sign-in, use the `DEV_SEED_PASSWORD` from `.env.demo` when using the demo database, or from `.env` when using the manual SQLite setup. The seed is blocked in production.

Development accounts for response authorities are scoped to the single sample jurisdiction **Tamil Nadu (`IN-TN`)**. This is a demo label only: the demo does not validate official state/district boundaries or route real emergency response. The production path intentionally refuses incident creation until jurisdiction routing is configured; enabling a real pilot requires approved Indian state/UT and district boundaries, agency authorization, policy sign-off, real integrations, and readiness review. Do not treat the development jurisdiction as a production default.

Try the workflow

1. Sign in as the citizen and submit a fictional incident.
2. Copy the tracking ID and private tracking key from the receipt.
3. Sign in as the dispatcher, review and prioritize the incident, then assign a responder.
4. Sign in as the responder, accept the assignment, record progress, and resolve it.
5. Sign in as the supervisor and close the incident.
6. Return as the citizen to view progress and submit feedback.
7. Review notifications and analytics; use the administrator role for the audit trail.

Connect the app to AWS

The code supports an AWS-backed path using Aurora PostgreSQL and optionally SQS, S3, SNS, Cognito, and CloudFront. AWS resources, permissions, network reachability, and account policy must all be configured first. The AWS showcase guide contains the full console procedure; the key connections are:

1. AWS identity and service permissions

Use the AWS CLI profile for the account that owns or can access the demo resources. For example:

```
aws login --profile resqcloud-demo --region ap-southeast-2
aws sts get-caller-identity --profile resqcloud-demo
```

The API's AWS SDK uses `AWS_PROFILE` to obtain temporary credentials for services such as SQS, S3, and SNS. Do not put AWS access keys in `.env`. Grant only the actions and resource ARNs the demo requires.

Organization accounts can restrict services. An AWS Organizations service control policy (SCP) can explicitly deny an action even when a role appears to have broad access. An account administrator cannot override an SCP deny; an organization administrator must review it, or the demo must use an account where the needed services are allowed.

2. Aurora PostgreSQL

In the repository root `.env`, set `DB_DIALECT=postgres`, the Aurora writer endpoint in `DATABASE_URL`, `DB_SSL=true`, and `DB_SSL_CA_PATH` to a local AWS RDS CA bundle. For the IAM authentication setup described in this repository, set `DB_IAM_AUTH=true`, configure `AWS_REGION`, and grant `rds-db:connect` to the exact database user and cluster resource ID.

The cluster must be **Available** and reachable from the machine running the API. If Aurora is stopped, the API fails during startup because it authenticates to the database before opening port 4000. If it is private, the app must run inside its network or use an approved private connection. A workstation cannot reach a private endpoint just by having AWS credentials.

Apply the schema and development records only after the database and IAM/network access are ready:

```
AWS_PROFILE=resqcloud-demo npm run db:migrate
AWS_PROFILE=resqcloud-demo npm run db:seed
AWS_PROFILE=resqcloud-demo npm run dev
```

3. Optional AWS integrations

- **SQS:** set `SQS_QUEUE_URL`; the API sends outbox events and polls/consumes queue messages. Configure the queue and dead-letter policy in AWS.
- **S3:** set `S3_BUCKET`; the API requests narrowly scoped AWS credentials for private evidence objects. Configure bucket CORS for `http://localhost:3000`.
- **SNS:** set `SNS_TOPIC_ARN` and confirm a subscription if the demonstration should deliver alerts.
- **Cognito:** configure the API-side pool/client identifiers plus the matching public variables `NEXT_PUBLIC_COGNITO_CLIENT_ID` and `NEXT_PUBLIC_COGNITO_DOMAIN`. Register `http://localhost:3000/auth/callback` as a callback URL. The app client is public and uses authorization code with PKCE; never put a Cognito client secret in browser variables.
- **CloudFront:** optional. Set the distribution domain, public key ID, and local private-key path. The S3 bucket stays private; the API signs short-lived viewer URLs after checking access.

`NEXT_PUBLIC_` *values are exposed in the browser bundle and are read when Next.js starts/builds. Restart the dev server after changing them; rebuild a production app. Keep database settings, private signing keys, and AWS credentials out of* `NEXT_PUBLIC_` variables.

Check the connection

```
curl http://localhost:4000/health
curl http://localhost:4000/ready
```

`/health` means the API process is answering. `/ready` also checks that the database can be reached and that the migration table exists. A successful health response alone does not prove Aurora is connected. Run `npm run lint`, `npm run typecheck`, `npm test`, and `npm run build` for local code checks. Browser tests use Playwright and require both local services unless `PLAYWRIGHT_BASE_URL` points at a running web app.

Common startup problems

Symptom	What it means / what to check

Port 3000 already in use	Another web process is running. Stop that process, then restart ResQCloud.
Port 4000 or `/ready` unavailable	The API may have failed its database startup check. Read the API log.
Aurora reports `stopped`	Start the cluster in RDS, wait for **Available**, then restart the local dev command.
`AccessDenied` mentions an SCP	Ask the organization administrator to review the explicit deny. Adding an IAM allow in the member account will not bypass it.
`unable to get local issuer certificate`	Check that `DB_SSL_CA_PATH` points to the correct, current CA bundle. Keep certificate verification enabled.
Connection timeout/reset	Check cluster state, endpoint/region/port, public or private reachability, security group rules, and local network access. AWS credentials do not create a network path into a VPC.
`NEXT_PUBLIC_*` config appears stale	Restart Next.js (or rebuild) after changing the value.

Project structure

Path	Responsibility
`apps/web/app/`	Next.js routes, page shells, and global styles.
`apps/web/features/`	Incident reporting, tracking, dashboards, admin, auth, and role views.
`apps/web/components/`	Shared UI and application-wide auth provider.
`apps/web/lib/`	Browser API client, Cognito flow, offline queue, and utilities.
`apps/api/src/`	Express routes, auth, authorization, database models/migrations, AWS adapters, and event worker.
`packages/shared/`	Shared TypeScript types, Zod schemas, and workflow rules.
`tests/`	API/domain/unit tests and Playwright browser tests.
`docs/`	AWS setup, architecture, security, status, and operational notes.

Useful commands:

```
npm run dev          # web :3000 + API :4000
npm run demo         # prepare SQLite seed data and start local demo
npm run demo:reset   # confirm, clear the local SQLite demo, reseed and start
npm run db:seed      # migrate and create fictional demo users/incidents
npm run db:migrate   # apply database migrations
npm run lint         # ESLint and Prettier checks
npm run typecheck    # TypeScript checks
npm test             # unit/domain/API tests
npm run build        # production API and web builds
npm run test:e2e     # Playwright browser tests
```

Project boundaries and demo safety

This is a functional local demo with AWS integration code; it is **not a production-ready emergency response system**. The integrations require account resources and live verification. Local tests use SQLite, not Aurora. Features such as S3 upload, CloudFront, Cognito, Maps, SQS and SNS do not become operational simply because their packages are installed; each needs valid service settings and permissions.

The current deployment guide does not configure AWS compute, load balancing, production secrets, horizontal worker coordination, or disaster recovery. Other documented production gates include MFA enrollment/recovery, evidence malware scanning, scale-out coordination, performance/security review, and operational recovery testing. Check implementation status before describing capabilities as complete.

Use fictional sample data only. Do not put real emergency, medical, or identifying information into this demo. Check current AWS pricing and account eligibility before creating resources; a budget alert notifies you but does not cap charges. After the demonstration, stop or delete resources as described in the AWS showcase cleanup steps.

Further reading

- Console-managed AWS setup
- Aurora PostgreSQL and IAM database authentication
- AWS architecture
- Application architecture
- API reference
- Security and privacy
- Implementation status and release gates
- Deployment boundary

SQS first AWS demo

Source: docs/aws/sqs-demo.md

SQS-first AWS demo setup

This guide is the low-cost first step for connecting a locally running ResQCloud API to one Amazon SQS Standard queue. For the project's complete AWS-backed application demo (RDS, SQS, S3 and SNS), use the AWS showcase guide. Both paths keep the web app and API running locally and avoid AWS compute infrastructure.

AWS free offers have account and usage conditions. SQS includes up to one million requests per month at no charge, but exceeding that limit can incur charges. This guide avoids S3, SNS, NAT Gateway, RDS and CloudFront in the first pass. If you need a hard $0 guarantee, do not create AWS resources; a budget alert is not a spending cap. Check your account's Free Tier status and SQS pricing before proceeding: AWS Free Tier, SQS pricing.

1. Install and verify the local app

Use Node.js 24 and npm. From the repository root:

```
cd /home/user/Projects/Cloud/resCloud
npm ci
node scripts/setupLocal.mjs
npm run db:seed
```

`setupLocal.mjs` creates a private `.env` configured for SQLite if one does not already exist. `db:seed` also runs the database migration and creates demo accounts and sample incidents. Keep `.env` private. Note the value of `DEV_SEED_PASSWORD` in that file; all seeded accounts use it.

2. Check AWS account and cost settings

1. Sign in to the AWS account you intend to use. Do not use the root user for

CLI work.

1. In **Billing and Cost Management**, review **Free Tier** and confirm which

offers apply to your account. Free account plan availability and limits depend on account eligibility.

1. If available, create a small AWS Budget alert (for example, `$1`) and email

notification. A budget is an alert, not a hard spending limit, and usage reporting can be delayed.

1. Choose one region and use it consistently. The project template defaults

to `ap-southeast-2` (Sydney); this guide uses that region.

3. Set up a restricted CLI identity in your personal account

Install or update to AWS CLI v2.32.0 or newer. You do not need IAM Identity Center, an organization, or an SSO start URL. In your personal AWS account, use the account owner's root sign-in in the **AWS Console for setup only**. Create a dedicated demo IAM user with console access; do not use the root identity to run the app. Attach the AWS-managed `SignInLocalDevelopmentAccess` policy so this identity can use browser-based CLI sign-in. You can do this in your own account; an organization is not required.

In the AWS Console:

1. Open **IAM** → **Users** → **Create user** and name it `resqcloud-demo`.
2. Enable AWS Management Console access for this user, set a unique password,

and enable MFA.

1. Attach the AWS-managed `SignInLocalDevelopmentAccess` policy. The

queue-specific policy in the next step grants the app its SQS access.

If your account's sign-in experience does not offer IAM users with console access, use a restricted console identity it provides. Do not create root access keys or run the app as root.

Then use browser login to obtain temporary credentials:

```
aws login --profile resqcloud-demo --region ap-southeast-2
aws sts get-caller-identity --profile resqcloud-demo
```

Complete the browser sign-in as the dedicated IAM user and check the returned account ID. AWS documents that `aws login` uses existing console credentials and refreshes temporary credentials. The identity also needs the SQS permissions in the next step. Do not create root access keys. AWS CLI browser login

If your CLI version does not support `aws login`, update it first. Avoid creating long-lived access keys as a workaround unless your account administrator explicitly requires that method. The AWS SDK in the API reads the CLI profile; no AWS keys belong in `.env`.

4. Create one SQS queue

In the AWS Console, open **Amazon SQS** → **Create queue**, select **Standard**, enter `resqcloud-demo-events`, and set **Receive message wait time** to 20 seconds and **Visibility timeout** to 60 seconds. Keep encryption at the default SQS-managed key; do not select a customer-managed KMS key for this demo. Create the queue and copy its URL from the queue details.

The API only needs `sqs:SendMessage`, `sqs:ReceiveMessage`, and `sqs:DeleteMessage` on this queue. Add these to the IAM user or role used by the profile, scoped to this queue's ARN. Get your account ID with `aws sts get-caller-identity --query Account --output text --profile resqcloud-demo`; the queue ARN is `arn:aws:sqs:ap-southeast-2:<ACCOUNT_ID>:resqcloud-demo-events`.

Add this inline policy to the demo IAM user, replacing `<ACCOUNT_ID>` with the account ID returned above:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["sqs:SendMessage", "sqs:ReceiveMessage", "sqs:DeleteMessage"],
      "Resource": "arn:aws:sqs:ap-southeast-2:<ACCOUNT_ID>:resqcloud-demo-events"
    }
  ]
}
```

In the AWS Console, open **IAM → Users → resqcloud-demo → Add permissions → Create inline policy → JSON**, paste the policy, replace the account ID, then review and save it. Re-run `aws login --profile resqcloud-demo --region ap-southeast-2` after changing permissions so the CLI refreshes the temporary session.

The API long-polls for up to 20 seconds and waits about one second before its next receive. While running continuously, that is roughly 124,000 receive requests in a 30-day month, below SQS's one-million-request free allowance before counting sends, deletes, or other account usage. A newly created incident can take up to about 21 seconds to appear in queue-driven notifications. With SQS disabled, local event delivery keeps its two-second poll interval.

5. Point the API at the queue

Open `/home/user/Projects/Cloud/resCloud/.env` and set:

```
AWS_REGION=ap-southeast-2
SQS_QUEUE_URL=https://sqs.ap-southeast-2.<ACCOUNT_ID>.amazonaws.com/<ACCOUNT_ID>/resqcloud-demo-events
```

Replace the placeholder URL with the exact `QueueUrl` from step 4. Leave `S3_BUCKET`, `SNS_TOPIC_ARN`, Cognito, and CloudFront settings blank for this first integration. No frontend variables or AWS credentials are needed.

6. Start the app with the AWS profile

From the repository root, authenticate and start both services:

```
cd /home/user/Projects/Cloud/resCloud
aws login --profile resqcloud-demo --region ap-southeast-2
AWS_PROFILE=resqcloud-demo npm run dev
```

Open `http://localhost:3000`; API health is at `http://localhost:4000/health` and readiness is at `http://localhost:4000/ready`. The API terminal should remain free of `AccessDenied` and SQS errors.

7. Verify the AWS-backed workflow

1. Sign in as `citizen@resqcloud.test` using `DEV_SEED_PASSWORD` from `.env`.
2. Create a test incident. Use sample details only; do not enter a real

emergency or personal information.

1. Let the dispatcher/responder workflow move the incident through a status

change, which writes an event to the transactional outbox.

1. Confirm the incident notification appears in the app. The backend sends

the outbox event to SQS, consumes it, and records an idempotent notification.

1. In the AWS Console, open **SQS → resqcloud-demo-events → CloudWatch metrics**.

Check sent and received message metrics; consumed messages are deleted, so the queue can correctly show zero visible messages. Metrics can take several minutes to appear. The console identity needs permission to view this queue and its CloudWatch metrics; ask the account administrator if needed.

1. Check the API log if notifications do not appear. `AccessDenied` usually

means the IAM identity lacks one of the three queue actions; a malformed queue URL or region mismatch also prevents delivery.

Run local checks any time with:

```
npm run typecheck
npm test
npm run build
```

8. Stop and delete the demo queue

Stop the app with **Ctrl+C**, then open **Amazon SQS → resqcloud-demo-events → Delete** in the AWS Console using an account administrator identity. Deleting the queue permanently removes messages. Remove the queue permissions from the demo IAM user if they were granted only for this demo, then sign out of its local CLI session:

```
aws logout --profile resqcloud-demo
```

Keep local SQLite data or delete it separately according to your needs.

Add other AWS services later

The API already supports private S3 uploads and SNS publishing, but these are not needed to demonstrate the first SQS integration. S3 can charge for storage, requests, or transfer; SNS delivery costs vary by endpoint. Check their account-specific offers and regional pricing before adding them. For the full console-managed showcase, including RDS, S3, SNS and optional Cognito, see AWS service showcase. To add only RDS first, see Aurora PostgreSQL.

Complete AWS service showcase

Source: docs/aws/service-showcase.md

Console-managed AWS showcase

This guide connects the locally running ResQCloud web app and API to AWS services created in your personal AWS account. The database is Aurora PostgreSQL; the other services are SQS, S3, SNS and optional Cognito. The app and API remain on your computer.

Service	Purpose	App configuration
Aurora PostgreSQL	Incidents, users, workflow history and notifications	`DB_DIALECT`, `DATABASE_URL`, `DB_SSL`, `DB_SSL_CA_PATH`, `DB_IAM_AUTH`
SQS	Outbox event delivery	`AWS_REGION`, `SQS_QUEUE_URL`
S3	Private incident evidence	`AWS_REGION`, `S3_BUCKET`
CloudFront (optional)	Signed delivery for private evidence	`CLOUDFRONT_DOMAIN`, `CLOUDFRONT_PUBLIC_KEY_ID`, `CLOUDFRONT_PRIVATE_KEY_PATH`
SNS	Critical incident email alerts	`AWS_REGION`, `SNS_TOPIC_ARN`
Cognito (optional)	Citizen registration and sign-in	Cognito pool/client/domain settings

Cost and account notes

Choose one AWS Region and use it for all resources; these steps use `ap-southeast-2` (Sydney), matching the current project resources. AWS currently offers Aurora PostgreSQL through **Create with express configuration** for eligible Free Tier accounts. That option is limited to 4 Aurora Capacity Units (ACUs) and 1 GB storage per cluster, and requires IAM database authentication. Check your account's offer and limits before creating the cluster. A full-configuration Aurora cluster may incur charges. A budget sends an alert; it does not cap spending. See Aurora Free Tier limits and Aurora pricing.

Express configuration uses an AWS-managed internet access gateway and IAM authentication. It does not use a VPC security group or password-based database login. Scope the runtime `rds-db:connect` permission to the exact cluster and DB username. For a full-configuration cluster, follow the RDS guide's security group and password-auth alternative, and check pricing first.

1. Prepare the local app and AWS CLI identity

From the repository root:

```
npm ci
node scripts/setupLocal.mjs
```

Protect your AWS account with MFA and do not create root access keys. Use a separate console identity with MFA for resource management. Create a dedicated IAM user named `resqcloud-demo` with console sign-in

and MFA for the local app profile, then attach the AWS-managed `SignInLocalDevelopmentAccess` policy. The runtime policy is added in step 6. An organization and IAM Identity Center are not required.

Install AWS CLI v2.32.0 or newer and sign in using browser-based console authentication:

```
aws --version
aws login --profile resqcloud-demo --region ap-southeast-2
aws sts get-caller-identity --profile resqcloud-demo
```

Verify the account ID belongs to you. The SDK uses temporary credentials from this profile; do not put AWS keys in `.env`. AWS's `aws login` setup and requirements are documented here.

2. Create Aurora PostgreSQL

In the AWS Console, choose `ap-southeast-2`, then open **RDS** → **Databases**. On the welcome page, find **Create with express configuration** and choose **Create**. Review the configuration, set a cluster identifier such as `resqcloud-demo`, and set the initial database name to `resqcloud`. Leave the internet access gateway and IAM authentication enabled. Create the database and wait until its status is **Available**.

From the cluster's **Connectivity & security** details, record:

- The writer/cluster endpoint hostname (do not include `https://` or `:5432`).
- Port `5432`.
- The cluster resource ID (used in the IAM policy).
- Your AWS account ID, available from the account menu or `aws sts get-caller-identity`.

Express configuration creates Aurora PostgreSQL with IAM database authentication and an internet access gateway. To connect to it, create an app DB role and grant the runtime IAM identity access. Follow the Aurora PostgreSQL guide for the DB role SQL and CA bundle setup.

If Express configuration is not available in your account, do not assume the full-configuration alternative is free. Either use local SQLite/PostgreSQL for the demo, or check the price estimate before creating a standard Aurora cluster. Full configuration needs a VPC and a carefully restricted security group for laptop access.

3. Create an SQS queue

Open **Amazon SQS** → **Create queue**:

1. Select **Standard** and name it `resqcloud-demo-events`.
2. Set **Receive message wait time** to 20 seconds and **Visibility timeout** to 60 seconds.
3. Keep SQS-managed encryption.
4. Create the queue and record its URL and ARN from the queue details.

4. Create a private S3 bucket

Open **Amazon S3** → **Create bucket**. Enter a globally unique name, choose `ap-southeast-2`, keep **Block all public access** enabled, and leave default encryption enabled. Create it and record the bucket name.

Open the bucket's **Permissions** → **CORS** settings and add this rule for local evidence uploads:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST", "HEAD"],
    "AllowedOrigins": ["http://localhost:3000"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

Do not add a public bucket policy. The API stores demo evidence under the `development/` key prefix.

5. Configure CloudFront for private evidence (optional)

CloudFront is optional; the authenticated API download continues to work without it. This adds a private CDN path: the API checks the user and signs a five-minute URL, CloudFront validates it with a trusted key group, and OAC lets CloudFront read the private S3 bucket. Do not enable public access on the bucket. CloudFront and data transfer can incur charges, so check current CloudFront pricing and your account's offer before creating a distribution. A budget alert does not cap spending.

Generate a signing key pair locally

From the repository root, create a 2048-bit RSA key pair. Keep the private key on this machine; only upload the public key to CloudFront. `.local-secrets/` is ignored by Git.

```
mkdir -p .local-secrets
openssl genrsa -out .local-secrets/cloudfront-private-key.pem 2048
openssl rsa -pubout -in .local-secrets/cloudfront-private-key.pem -out .local-secrets/cloudfront-public-key.pem
```

Do not paste or commit the private key. The API reads it from the path configured in `.env`.

Add the public key and trusted key group

1. In **CloudFront → Public keys**, choose **Create public key**. Give it a name such as `resqcloud-demo-signer`, open `cloudfront-public-key.pem` locally, and paste its full PEM contents into the public key field. Save and record the **Public key ID** (not the key group ID).
2. In **CloudFront → Key groups**, choose **Create key group**, add that public key, save, and record the group name. CloudFront recommends trusted key groups for signed URLs. See AWS's trusted signer guide.

Create a distribution with a private S3 origin

1. Open **CloudFront → Distributions → Create distribution**. Choose the evidence S3 bucket as the origin. Use its normal regional S3 bucket endpoint, not the S3 website endpoint.
2. Set **Origin access** to **Origin access control settings (recommended)**. Create an OAC with **Sign requests (recommended)**. Keep all S3 Block Public Access settings enabled.
3. For the default cache behavior, redirect HTTP to HTTPS; allow only **GET, HEAD**; use the default cache policy; and set **Restrict viewer access** to **Yes**. Choose **Trusted key groups** and add the key group created above.

4. Create the distribution. When CloudFront offers to update the bucket policy for OAC, allow it. Otherwise, apply the policy CloudFront provides under the origin's access settings. Check that the policy grants `cloudfront.amazonaws.com` access only for this distribution and the bucket objects. Do not add anonymous `s3:GetObject` access.
5. Wait for the distribution status to become **Deployed**. Record its **Distribution domain name**, for example `d123example.cloudfront.net`. The default CloudFront hostname already supports HTTPS; no custom domain or certificate is needed for this demo.

OAC secures the origin connection, while the trusted key group restricts viewers. Both are required for private signed delivery. AWS describes the supported OAC setup here and the signed URL flow here.

6. Create an SNS topic

Open **Amazon SNS** → **Topics** → **Create topic**, select **Standard**, and name it `resqcloud-critical-alerts`. Create the topic and record its ARN. Create an **Email** subscription for an address you can access, then confirm the subscription from the message SNS sends.

7. Grant the runtime identity narrow permissions

In **IAM** → **Users** → **resqcloud-demo** → **Add permissions** → **Create inline policy** → **JSON**, paste this policy. Replace each placeholder with the exact value from the console. The cluster resource ID appears in Aurora cluster details. The `rds-db:connect` ARN must include the database username `resq_app` at the end.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["sqs:SendMessage", "sqs:ReceiveMessage", "sqs:DeleteMessage"],
      "Resource": "REPLACE_WITH_QUEUE_ARN"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::REPLACE_WITH_BUCKET_NAME/development/*"
    },
    {
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "REPLACE_WITH_TOPIC_ARN"
    },
    {
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:ap-southeast-
2:REPLACE_WITH_ACCOUNT_ID:dbuser:REPLACE_WITH_CLUSTER_RESOURCE_ID/resq_app"
    }
  ]
}
```

Name the policy `ResQCloudDemoRuntime`, review it, and save. The `rds-db:connect` ARN uses the **cluster resource ID**, not the endpoint, cluster identifier, or ordinary RDS ARN. AWS documents the required ARN format here.

8. Optionally configure Cognito

For hosted citizen sign-in, open **Amazon Cognito → User pools → Create user pool** and configure email as a sign-in option. Create an app client as a **public client** with no client secret. Enable authorization code grant with PKCE; allow scopes `openid`, `email`, and `profile`. Register these URLs exactly:

- Callback: `http://localhost:3000/auth/callback`
- Sign out: `http://localhost:3000`

Create a Cognito domain. Record the user pool ID, app client ID, and full domain URL. Cognito's public client should use authorization code with PKCE; see app client settings. You can skip Cognito and use the local seeded accounts.

9. Configure `.env`

Use the private `.env` created by `setupLocal.mjs`. Download the Aurora/RDS CA bundle from the project root:

```
mkdir -p .local-secrets
curl -fsSL https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem -o .local-secrets/rds-global-bundle.pem
curl -fsSL https://www.amazontrust.com/repository/AmazonRootCA1.pem -o .local-secrets/amazon-root-ca-1.pem
cat .local-secrets/rds-global-bundle.pem .local-secrets/amazon-root-ca-1.pem > .local-secrets/aurora-ca-bundle.pem
realpath .local-secrets/aurora-ca-bundle.pem
```

Set these values in `.env`, replacing the example endpoint, bucket, queue URL, topic ARN, account ID and cluster resource ID in the policy where applicable:

```
AWS_REGION=ap-southeast-2
DB_DIALECT=postgres
DATABASE_URL=postgres://resq_app@YOUR_AURORA_WRITER_ENDPOINT:5432/resqcloud
DB_SSL=true
DB_SSL_CA_PATH=/absolute/path/to/resCloud/.local-secrets/aurora-ca-bundle.pem
DB_IAM_AUTH=true
SQS_QUEUE_URL=YOUR_QUEUE_URL
S3_BUCKET=YOUR_BUCKET_NAME
SNS_TOPIC_ARN=YOUR_TOPIC_ARN
CLOUDFRONT_DOMAIN=YOUR_DISTRIBUTION_DOMAIN
CLOUDFRONT_PUBLIC_KEY_ID=YOUR_CLOUDFRONT_PUBLIC_KEY_ID
CLOUDFRONT_PRIVATE_KEY_PATH=/absolute/path/to/resCloud/.local-secrets/cloudfront-private-key.pem
```

If CloudFront is not configured, leave all three `CLOUDFRONT_*` values blank. Do not set `AWS_ENDPOINT` for AWS; leave it blank so the AWS SDK uses the standard regional service endpoints. `S3_BUCKET` is the bucket name only (for example `resqcloud-evidences-2026`), not its ARN or URL.

The app generates a short-lived Aurora IAM database token when it opens a PostgreSQL connection. Do not add a DB password to this URL. Keep the database endpoint, Region, username, and `rds-db:connect` policy values consistent. If Cognito is enabled, also set `COGNITO_USER_POOL_ID`, `COGNITO_CLIENT_ID`, `NEXT_PUBLIC_COGNITO_CLIENT_ID`, and `NEXT_PUBLIC_COGNITO_DOMAIN` to the matching values.

10. Migrate, seed, and run

From the repository root, create the schema and fictional demo records in Aurora:

```
npm run db:migrate
npm run db:seed
```

Make sure `NODE_ENV=development` remains set. Read `DEV_SEED_PASSWORD` from `.env`. Authenticate the profile and start the app in two separate commands:

```
aws login --profile resqcloud-demo --region ap-southeast-2
AWS_PROFILE=resqcloud-demo npm run dev
```

Open `http://localhost:3000`. API health is `http://localhost:4000/health`; dependency readiness is `http://localhost:4000/ready`.

Demonstrate the services using fictional data: update an incident to send/consume an SQS event, upload a small evidence image and check for its private S3 object, request the image through its signed CloudFront URL, and make a test incident critical to trigger SNS email. A URL is valid for five minutes. To verify privacy, try the S3 object URL directly; it should be denied. SQS messages may be gone by the time you inspect the queue because the API consumes and deletes them; compare sent and received metrics. Seeded critical events may also send email alerts. If Cognito is configured, complete the citizen sign-up and email confirmation flow.

11. Clean up after the demo

Stop the local app. In the AWS Console, disable and delete the CloudFront distribution (deletion is allowed after it is disabled), delete its key group and public key, empty and delete the S3 bucket, delete the SQS queue and SNS topic, and delete the Cognito user pool if created. Delete the Aurora cluster and its DB instances; if the data is disposable, do not retain a final snapshot. Remove the runtime inline policy and demo IAM user if no longer needed. Review Billing for remaining snapshots or other resources.

For an SQS-only first setup, see the low-cost AWS guide. This project does not deploy the web app or API to AWS compute; see deployment status.

Aurora PostgreSQL

Source: docs/aws/aurora-postgresql.md

Connect ResQCloud to Aurora PostgreSQL

The project uses Sequelize with PostgreSQL for its Aurora database connection. The recommended console path is Aurora PostgreSQL **Create with express configuration**, which uses IAM database authentication and the Aurora internet access gateway. The API creates short-lived database authentication tokens from the local AWS CLI profile; no database password is stored in `.env`.

Aurora Free Tier and express configuration

AWS lists Aurora PostgreSQL under its Free Tier through express configuration for eligible accounts. Express clusters have an AWS-managed internet access gateway, use IAM authentication only, and currently have limits of up to 4 ACUs and 1 GB of storage per cluster. These limits and any credits are account-plan dependent. Full configuration has different options and pricing, and should not be assumed free. See Aurora Free Tier limitations and express configuration settings.

1. Create Aurora PostgreSQL in the Console

1. Open **Amazon RDS** → **Databases** and select the AWS Region for the demo (the project examples use `ap-southeast-2`).
2. On the RDS welcome page, select **Create with express configuration** → **Create**.
3. Set a cluster identifier such as `resqcloud-demo` and initial database name `resqcloud`.
4. Keep IAM database authentication and the internet access gateway enabled.
5. Review the selected account plan and settings, create the database, and wait until its status is **Available**.
6. From cluster details, record the writer/cluster endpoint, port `5432`, and **cluster resource ID**. The resource ID is different from the endpoint and cluster identifier.

Express configuration does not use a VPC security group; its managed gateway supports internet access with IAM authentication. Do not switch to full configuration without reviewing the cost and networking requirements.

2. Create the app database role

The express cluster's administrator uses IAM authentication. From the Aurora cluster's **Connectivity & security** tab, use the RDS Console's **CloudShell** connection option to open `psql` as the administrator. Select the `resqcloud` database, then run:

```
CREATE ROLE resq_app LOGIN;
GRANT rds_iam TO resq_app;
GRANT CONNECT ON DATABASE resqcloud TO resq_app;
GRANT USAGE, CREATE ON SCHEMA public TO resq_app;
```

This gives the app's migration role permission to create its tables in the `public` schema. The app identity will still be restricted by the IAM `rds-db:connect` ARN in the showcase guide. AWS describes creating PostgreSQL users for IAM authentication here.

3. Set up the RDS CA bundle

From the project root, download the AWS global database CA bundle:

```
mkdir -p .local-secrets
curl -fsSL https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem -o .local-secrets/rds-global-bundle.pem
curl -fsSL https://www.amazontrust.com/repository/AmazonRootCA1.pem -o .local-secrets/amazon-root-ca-1.pem
cat .local-secrets/rds-global-bundle.pem .local-secrets/amazon-root-ca-1.pem > .local-secrets/aurora-ca-bundle.pem
realpath .local-secrets/aurora-ca-bundle.pem
```

The combined trust file contains AWS's RDS roots and Amazon Root CA 1. This Aurora endpoint currently presents a certificate chain rooted at Amazon Root CA 1, so using only the RDS global bundle fails certificate verification. The API continues to verify the TLS chain. AWS documents its RDS CA bundles here; Amazon Trust Services recommends including its root certificates in custom trust stores here.

4. Configure the API

In the root `.env`, set the Aurora writer endpoint, database role, and absolute CA file path:

```
AWS_REGION=ap-southeast-2
DB_DIALECT=postgres
DATABASE_URL=postgres://resq_app@YOUR_AURORA_WRITER_ENDPOINT:5432/resqcloud
DB_SSL=true
DB_SSL_CA_PATH=/absolute/path/to/resCloud/.local-secrets/aurora-ca-bundle.pem
DB_IAM_AUTH=true
```

Do not put a password in this URL for an express cluster. `DB_IAM_AUTH=true` causes the API to generate a temporary token for each new PostgreSQL connection using the AWS SDK signer and credentials selected by `AWS_PROFILE`.

The database URL must use the writer/cluster endpoint, not a custom DNS alias, because IAM token signatures are made for the exact endpoint. Keep the Region, endpoint, port, DB role, and IAM policy in sync.

5. Run migrations and seed data

After adding the `rds-db:connect` permission for `resq_app` to the `resqcloud-demo` IAM identity, log in and apply the schema:

```
aws login --profile resqcloud-demo --region ap-southeast-2
npm run db:migrate
npm run db:seed
```

The seed command creates fictional dispatcher, responder, supervisor, admin, and citizen accounts and sample incidents. It requires `NODE_ENV=development`; the generated seed password is in `.env`. Then start the app:

```
AWS_PROFILE=resqcloud-demo npm run dev
```

Check `http://localhost:4000/ready` for dependency readiness and open `http://localhost:3000` for the app.

If migration fails with `permission denied for schema public`, connect to the `resqcloud` database as its administrator in RDS Console CloudShell and grant the dedicated app role the required schema permissions:

```
\c resqcloud
GRANT USAGE, CREATE ON SCHEMA public TO resq_app;
SELECT has_schema_privilege('resq_app', 'public', 'USAGE') AS can_use,
       has_schema_privilege('resq_app', 'public', 'CREATE') AS can_create;
```

The app role needs permission to create and own its tables, but should not be a database administrator.

If you choose Aurora full configuration instead

Full configuration can use password authentication. It is not the express Free Tier path. Create Aurora PostgreSQL with a VPC, public access only if needed for the laptop, and a security group allowing TCP `5432` from your current public IP `/32` only. Never use `0.0.0.0/0`. Create a separate database user and grant it access to `resqcloud`; do not run the app as the master user.

For that password-auth setup, set `DB_IAM_AUTH=false` and use a password-bearing URL, for example:

```
DB_DIALECT=postgres
DATABASE_URL=postgres://resq_app:URL_ENCODED_PASSWORD@YOUR_AURORA_WRITER_ENDPOINT:5432/resqcloud
DB_SSL=true
DB_SSL_CA_PATH=/absolute/path/to/resCloud/.local-secrets/aurora-ca-bundle.pem
DB_IAM_AUTH=false
```

URL-encode special characters in the password. Keep `DB_SSL=true` and use the CA bundle for either configuration.

Delete the demo database

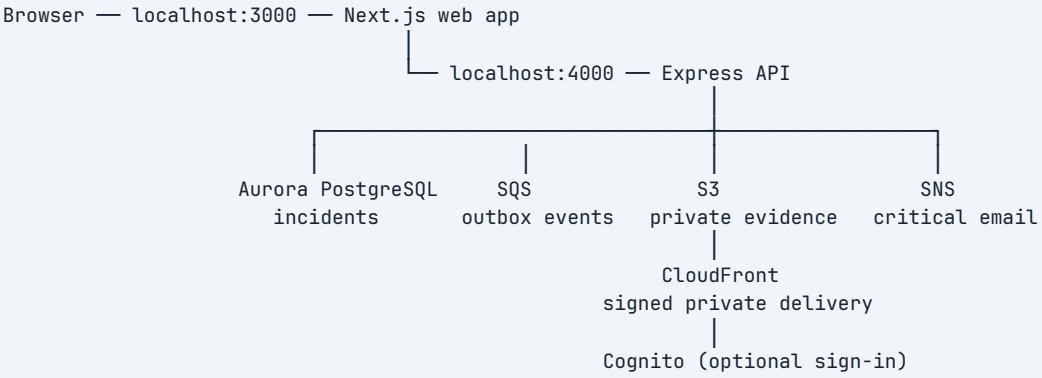
Stop the app. In **RDS → Databases**, delete the Aurora DB instances and cluster. Do not retain a final snapshot if the data is disposable; snapshots may continue to incur charges. Remove the `rds-db:connect` permission and `resq_app` role if they are no longer needed, then check Billing for remaining resources.

AWS architecture

Source: docs/aws/architecture.md

AWS showcase architecture

The demo web app and API run locally. AWS services are created and managed directly in the AWS Console in the user's personal account.



The API uses a local AWS CLI profile for SQS, S3 and SNS. Aurora PostgreSQL access uses a dedicated database role, verified TLS, and short-lived IAM database tokens. Cognito is an optional browser sign-in integration. SQLite and local password sign-in remain available for development and tests.

CloudFront is optional and serves confirmed evidence through five-minute signed URLs. S3 remains private: CloudFront uses Origin Access Control (OAC), while a trusted key group verifies viewer URLs. The API signs URLs locally with the corresponding private key; the private key is not uploaded to AWS.

This setup does not include AWS-hosted application compute, a load balancer, NAT Gateway, WAF, CloudTrail configuration or centralized application logs. It is a faculty demo architecture, not a production deployment; see deployment.md and STATUS.md.

Deployment boundary

Source: docs/engineering/deployment.md

Deployment boundary

ResQCloud is currently a locally run application with optional connections to AWS-managed services created in the AWS Console. The app and API are not deployed to AWS. The console-managed demo setup is documented in the AWS service showcase.

There is no Terraform configuration or AWS compute deployment in this repository. Hosting the web app and API on AWS is future work and would need a separate design for compute, networking, secrets, TLS, deployment automation, scaling and recovery. Do not treat the local service integration as a production deployment.

Before presenting, run the local production build and application checks documented in the README, verify the configured AWS resources in their service consoles, and check `/ready`. Do not add real emergency or personal data to the demo.

Implementation status and production gates

Source: docs/project/status.md

Implementation status

This handoff delivers a functional local application and AWS integration code. It does **not** certify completion of every phase in the supplied 62-section implementation plan. No production resources were created.

Implemented in the application

Area	Delivered behavior
Foundation	Next.js/Express TypeScript workspaces; Tailwind; reusable Radix/shadcn-style controls; Zod; React Hook Form; lint/format/typecheck/build commands
Data	Versioned Sequelize migrations through `004_incident_timing`, foreign keys and indexes; core tables, sessions, media, outbox and consumed-event records
Authentication	Development citizen registration; authority password login; bcrypt; short JWTs; rotating refresh cookies; reuse revocation; sessions; TOTP verification; disabled-user checks
Authorization	Citizen ownership, department scope, responder assignment and admin checks on protected APIs
Reporting	Anonymous or signed-in reporting, coordinates, category, severity, description, affected count, incident time and private tracking receipt
Workflow	Controlled status changes, reasons and history; assessment; escalation/reopening; responder recommendations, acceptance, assignment and resolution; supervisor closure
Privacy	Tracking secret hash; public allowlist projection; approximate public coordinates; anonymous contact suppression; no secrets in audit payloads
Operations	Queue search/filter/page controls, detail actions, duplicate candidates, notifications/read state, department responder lists, location recording
Reliability	Transactional outbox, backoff, local dead-event retries, SQS producer/consumer, idempotent in-app notifications, WebSocket user rooms and polling fallback
SOS/offline	Explicit SOS confirmation, critical reporting; encrypted signed-in report drafts with expiry and retry
Feedback/analytics	Reporter-only feedback after closure, uniqueness guard, scoped SQL counts/category/status groups, UTC date-filtered analytics/CSV export, average first-response/resolution durations
Administration	Create authorities, enable/disable accounts, change role/department/jurisdiction, revoke sessions, department listing, audit viewing and failed event retry

Implemented but requiring configured services and live verification

Cognito hosted sign-in/registration/recovery with PKCE; S3 evidence upload/confirmation/download; CloudFront signed access; Mapbox incident markers; SQS delivery/retries/DLQ; SNS delivery; Aurora PostgreSQL runtime with IAM database tokens; console-managed AWS resources.

Local tests use SQLite, **not Aurora PostgreSQL**. Browser tests exercise local login and report tracking. AWS integration code is not evidence that those services have been deployed or tested.

India jurisdiction slice (development demo)

The current development interface exposes one sample state, Tamil Nadu (`IN-TN`). A versioned migration adds jurisdiction codes to users and incidents, scopes operations data and notification delivery by state as well as department, and prevents assigning a responder from a different state. Production's state selector catalog includes all 28 states and 8 union territories, but this is only a names/codes catalog: districts, approved boundaries, geocoding, agency mappings, and policy are not configured. Production incident creation therefore fails closed until routing is implemented and approved. See the regional readiness audit for the requirements map and pilot gates.

Remaining before the original full production definition of done

- Complete the missing `AGENT.md` policy review, including operating jurisdictions, retention, SLA definitions and incident-specific forms.
- Production MFA enrollment, encrypted MFA-secret storage, recovery codes, trusted provisioning and authority account recovery.
- Full category/system configuration management and its UI. Admin role/department/jurisdiction reassignment is implemented with session revocation and protection for active responder assignments.
- Map pin selection, live responder map layers, routes, coverage polygons, incident radii, heatmaps and density history.
- Automated response/resolution time calculations, configurable SLA breaches, geographic analytics, PDF and asynchronous export jobs. Date-filtered analytics and CSV exports are implemented.
- Notification channel preferences and complete email/SMS/push subscription flows. SNS topic publishing alone does not implement these experiences.
- Evidence malware/content-signature scanning, quarantine, abandoned upload cleanup, anonymous evidence upload and image/video previews.
- Offline attachment sync, draft-management UI, retry-conflict resolution and service-worker cold-start support.
- Horizontal scaling: distributed rate limits, cross-instance WebSocket delivery, event-worker ownership/locking and rolling-upgrade concurrency testing. The current configuration intentionally runs one application instance.
- Configurable SLA breach targets remain absent. Average response/resolution durations are available, but they are operational measurements rather than policy thresholds.
- Timing averages include incidents whose response/resolution milestones were recorded after migration `004_incident_timing`; older incidents are not backfilled and are excluded from the sample count.

- Automated image similarity is absent; duplicate suggestions use distance, category, time and description overlap and never auto-merge.
- Full browser lifecycle coverage across every role, real Aurora PostgreSQL integration tests, cloud failure injection, load/performance budgets, screen-reader review and penetration testing.
- Production image baking, OIDC deployment roles, environment approval gates, real rolling deployment automation, configured monitoring dashboards, backups and a tested disaster-recovery rehearsal.

The supplied requirements and implementation plan are maintained in implementation-plan.md. This status file is the authoritative handoff boundary; pending work is not represented as completed functionality.